

Multi-Agent Reinforcement Learning for Coordinating Communication and Control

Federico Mason¹, *Member, IEEE*, Federico Chiariotti², *Member, IEEE*,
Andrea Zanella¹, *Senior Member, IEEE*, and Petar Popovski³, *Fellow, IEEE*

Abstract—The automation of factories and manufacturing processes has been accelerating over the past few years, leading to an ever-increasing number of scenarios with networked agents whose coordination requires reliable wireless communication. In this context, goal-oriented communication adapts transmissions to the control task, prioritizing the more relevant information to decide which action to take. Instead, networked control models follow the opposite pathway, optimizing physical actions to address communication impairments. In this work, we propose a joint design that combines goal-oriented communication and networked control into a single optimization model, an extension of a multi-agent Partially Observable Markov Decision Process (POMDP), which we call Cyber-Physical POMDP. The proposed model is flexible enough to represent a large variety of scenarios and we illustrate its potential in two simple use cases with a single agent and a set of supporting sensors. Our results assess that the joint optimization of communication and control tasks radically improves the performance of networked control systems, particularly in the case of constrained resources, leading to implicit coordination of communication actions.

Index Terms—Markov decision processes, networked control systems, goal-oriented communications, multi-agent reinforcement learning.

I. INTRODUCTION

OVER the last decade, the use of Internet of Things (IoT) technology for industrial and manufacturing applications has exploded [1]. The digitization and automation of industrial processes is a key component of the Industry 4.0 paradigm [2], and the coordination of mobile robots requires the use of wireless communications. The optimization of these systems has been approached from two opposite directions: from the control side, we have seen the development of networked control systems [3] which account for the Quality of Service

(QoS) using metrics such as the latency distribution or the packet error rate. From the communication perspective, growing attention has been dedicated to new performance indicators that can capture the communication needs of control applications, guiding the optimization of scheduling and resource allocation [4]. A representative indicator is Age of Information (AoI) [5], which measures the freshness of the information available to the controller.

Another significant development in the field has been the explosion of Reinforcement Learning (RL) techniques, which began in 2015 with DeepMind’s seminal paper on Deep Q-Networks (DQNs) [6]. The union of deep learning with reinforcement techniques has triggered a revolution in robotics, as DQNs are more flexible and adaptable than human-designed techniques in a variety of complex situations. Naturally, the DQN is optimized for its training scenario, and will effectively act as a networked control system, countering the impairments caused by the wireless connection and maximizing performance, if it is trained in that environment. However, the inherent complexity of learning models leads to significant challenges: since learning agents operate as black boxes, it is difficult to know which piece of information will be crucial and which will be almost irrelevant [7], reducing the effectiveness of AoI-based approaches.

The quest for sending relevant and meaningful information has led to an increased interest in semantic communications. These approaches are commonly inspired by Weaver’s introduction to the expanded version of Shannon’s seminal paper [8]. According to this text, the optimization of a set of QoS measures may involve two distinct problems, i.e., a *semantic* and an *effectiveness* problem, respectively. Drawing a full parallel to linguistics, effectiveness corresponds to *pragmatics*, where the ultimate goal of communication is to affect the behavior of intelligent agents, taking into account shared context and implicit clues. While effectiveness has been discussed for decades [9], its complexity has limited the possibility of practical solutions until the massive improvement of machine learning techniques. However, this research domain is still in its infancy, and concurrently learning to control a system and communicate efficiently is an open problem for non-trivial scenarios [10].

In this paper, we propose a general model of a Cyber-Physical System (CPS) that integrates communication and control, allowing for their joint optimization using Multi-Agent Reinforcement Learning (MARL) [11]. More specifically, we consider a dynamic environment in which

Manuscript received 17 October 2023; revised 30 January 2024 and 12 March 2024; accepted 23 March 2024. Date of publication 3 April 2024; date of current version 8 August 2024. This work was partly supported by the European Union under the Italian National Recovery and Resilience Plan (NRRP) of NextGenerationEU, as part of the REDIAL Young Researchers grant (SoE0000009) and the RESTART partnership on “Telecommunications of the Future” (PE0000001). The work of Petar Popovski was supported by the Villum Investigator Grant “WATER” from the Velux Foundation, Denmark. The associate editor coordinating the review of this article and approving it for publication was L. Liang. (*Corresponding author: Federico Mason.*)

Federico Mason, Federico Chiariotti, and Andrea Zanella are with the Department of Information Engineering, University of Padova, 35131 Padua, Italy (e-mail: masonfed@dei.unipd.it; chiariot@dei.unipd.it; zanella@dei.unipd.it).

Petar Popovski is with the Department of Electronic Systems, Aalborg University, 9220 Aalborg, Denmark (e-mail: petarp@es.aau.dk).

Digital Object Identifier 10.1109/TCCN.2024.3384492

multiple agents interact. There are two agent types: *robots*, which can perform actions and affect the physical environment; and *sensors*, which communicate their observations to the robots through resource-constrained communication channels. This maps directly to the effectiveness problem, as the goal is to optimize the control of the robots by exploiting data from the sensors to maintain an accurate picture of the state of the system. Our model follows the Partially Observable Markov Decision Process (POMDP) theory, and the joint communication and control aspects, as well as the interaction between agents, led us to name it Cyber-Physical POMDP (CP-POMDP). Some existing models, such as Networked Distributed POMDPs (ND-POMDPs), include communications between agents but do not optimize them directly. By explicitly modeling communication decisions, and defining when and between whom communication happens as part of the optimization, CP-POMDPs represent a subset of possible ND-POMDPs in which communication is decided by the agents as part of their policies, particularly suited for CPSs.

Our model captures the direct and indirect aspects of communication, as sharing information can affect both the physical environment and the observations available to each agent about it. More importantly, control and communication decisions are both captured by the model, allowing the joint training of agents that perform one or both types of actions, i.e., interact with the physical environment and communicate local information to other nodes. Previous approaches in effective communication [10], [12], [13] optimized communication parameters such as source and channel coding or packet scheduling for a fixed control strategy, while a common approach on the control side is to optimize the control strategy with given communication QoS parameters [14], [15]. Conversely, our solution bridges the gap between the communication and control approaches, combining them into a single comprehensive MARL model by following the principle that *coordination is implicit communication* [16], that is, the line between acts of communication and acts of control is blurred [17] and the two aspects should be considered and optimized jointly. With respect to existing MARL models, our formulation is specifically focused on this dual nature of communication, and explicitly optimizes communication and control as two distinct but interconnected parts of the system.

The benefits of solving the effectiveness problem are more pronounced when the communication is constrained and the data that gets actually transmitted should be carefully selected. This is typical in underwater environments with acoustic communications [18], which are used in this paper to showcase our MARL solution. Particularly, we consider two different scenarios where an Autonomous Underwater Vehicle (AUV) needs to reach a surface vessel as fast as possible, and static sensor buoys can expand the AUV's field of view by communicating local observations. In the first scenario, the AUV's movements are constrained by obstacles, such as underwater installations natural rock formations, and even sea mines. In the second scenario, the AUV has to perform a data muling mission, recovering data from floating sensors that might move with the current [15]. Our results show that joint optimization can significantly improve the performance

of the AUV both when the communication decisions are made by a centralized controller with full knowledge of the map and when they are made in a distributed fashion by the individual buoys. The implications of the proposed method are not limited to these simple underwater models, and we provide some examples of larger and more complex CP-POMDPs.

The rest of this paper is organized as follows: first, we present the state of the art on AoI optimization and networked control systems in Section II, then define the CP-POMDP model in Section III. We then focus on the case with a single robot, aided by a certain number of sensors: the system model is rigorously defined as an instance of a CP-POMDP in Section IV. The two underwater tasks and the simulation settings and results are presented in Section V, and finally, Section VI concludes the paper and lists some possible avenues of future work on the subject.

II. RELATED WORK

The growing use of wireless devices in control and monitoring scenarios with significant resource constraints, such as industrial automation, coordination between autonomous cars, and drone swarm management, has led to the definition of the AoI metric [19]. Unlike packet latency, which measures the time from a packet's generation to its delivery, the AoI represents the freshness of the information available to the receiver, accounting for both the latency and the rate at which the source generates data [5]. AoI has become widespread due to its intuitive relation with control processes: the freshness of the information when a control agent makes a decision represents the delay between the environment estimation and the decision execution. Since its first introduction, there has been a plethora of works computing AoI in different scenarios and optimizing communications to minimize the average or worst-case AoI [20].

However, AoI does not necessarily convey enough information to optimize intelligent networked control systems. Frequent updates are wasteful if the monitored process remains stable while, if the process sharply changes, new information becomes urgently relevant even if an update has just been transmitted. In Markovian scenarios, this issue is solved by using the Age of Incorrect Information (AoII) [21], which extends the notion of age by considering only the state updates carrying new information. A similar approach can be applied to non-Markovian processes, leading to the more general concept of Value of Information (VoI) [7]: this metric is essentially an error function that depends on the difference between the estimated state at the receiver and the actual state of the process [22]. However, even VoI might not be the optimal choice for networked control systems, as it is not ensured that estimation errors translate to lower performance. If the controller is highly sensitive only to some error types, defining the error function becomes complicated, and it is better to just use control performance as a proxy. This vision leads to the Urgency of Information (UoI) metric [12], which directly considers how much an update would affect a (known) controller.

A common assumption for the optimization of AoI, VoI, and UoI metrics is to keep a fixed control strategy. Instead, the work on networked control systems goes in the opposite direction, assuming a fixed communication strategy and varying the control decisions. The most common approach is to model a limited number of QoS metrics and design a controller that can perform well within a given operating region. Delay is the most common impairment that these systems consider [14], considering also the difference in the uplink and downlink directions [23]. Another important parameter is packet loss, as the erasure of an update can significantly affect the controller's decisions [24]. Communication failures can be addressed by using reliable communication protocols, which re-transmit lost packets, or Forward Error Correction (FEC), trading additional latency [25].

Age and packet loss are not the only issues for the management of networked control systems. Another critical constraint is given by the channel capacity, especially in the case of sensors transmitting video or other feature-rich data. In this case, the communication system needs to choose which pieces of information to transmit, and at which compression level [26], avoiding channel congestion. Data compression, expressed in terms of quantization [27] or partial information, may critically affect the controller's decisions [28]. In such a context, it is advantageous to adopt Machine Learning (ML) approaches as their generalization capabilities enable them to deal with unforeseen situations and compensate for the network impairments also in non-linear systems [29]. Hence, ML proved to be a viable option for network control optimization both in case of random delays [30] and compressed data [31].

The management of networked control systems has also been already approached in the MARL literature, either for the optimization of the communication system or for dealing with imperfect and delayed information. One of the first works to do so, by Foerster et al. [32], considers the problem of coordinating multiple agents, showing emergent communication capabilities. Other works use explicit or implicit communication costs so that agents can choose whether to coordinate or act independently [33]. However, to the best of our knowledge, no unified framework for learning in networked control systems has been proposed.

Two recent works are the most similar to our own: in the first [13], multiple agents learn the parameters of a simple scheduling algorithm while performing a reinforcement task in real-time. However, the use of a simple scheduling algorithm limits the maximum performance of the solution, while our work uses the Decentralized POMDP (Dec-POMDP) framework makes it possible to fully explore the system environment and identify new and more efficient solutions. The second one [10] considers a scenario similar to our own, with one sensor and one robot, which design a shared codebook instead of learning which information to transmit. Particularly, this work could be combined with ours to form a more complete communication and control system, as it focuses on encoding the available information instead of scheduling which part of it is more important in a distributed setting. For a more thorough review of the cooperative MARL literature, we refer the reader to [34].

III. JOINT COMMUNICATION AND CONTROL

The problem addressed in this paper can be seen as a special case of the more general class of Dec-POMDPs. While Dec-POMDPs considers communication implicitly, our model gives an explicit structure to the communication process among the system agents. We name our model CP-POMDP, as agents can both act in a physical environment and exchange information with other agents, coordinating to achieve a common goal.

A. Reinforcement Learning

In Markov Decision Processes (MDPs), time is discretized into slots so that, at each slot k , the environment is fully described by a state $s(k) \in \mathcal{S}$. In particular, an agent periodically observes the environment state and takes actions accordingly, affecting the system state evolution. Hence, at each slot k , the agent performs a new action $a(k) \in \mathcal{A}$, and receives a reward $r(k) \in \mathbb{R}$, depending on both $s(k)$ and $a(k)$. The agent behavior follows a policy $\pi : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$, where $\pi(s, a)$ is the probability to take action $a \in \mathcal{A}$ given the state $s \in \mathcal{S}$. If the environment is only partially observable, the agent perceives the state through a function $\mathcal{S} \rightarrow \mathcal{O}$, associating each state s with an observation o . The domain of the policy function is then given by the belief space $\mathcal{B}$ over $\mathcal{S}$, i.e., the set of probability measures over $\mathcal{S}$, representing the possible *a posteriori* distributions of the state given the history of past observations. The goal of any RL algorithm is to find a policy π^* that maximizes the agent *discounted return*, defined as

$$G(k) = \sum_{\nu=0}^{\infty} \lambda^{\nu} r(k + \nu), \quad (1)$$

where $\lambda \in [0, 1]$ is an exponential *discount factor*. To achieve this goal, RL needs to estimate the value of each possible belief-action pair $(b, a) \in \mathcal{B} \times \mathcal{A}$, determining the best action for each observation. In other words, given a policy π , the algorithm approximates the Q-function

$$Q_{\pi}(b, a) = \mathbb{E}[G(k) | b(0) = b, a(0) = a, \pi], \quad (2)$$

which is the expectation of the return obtained by choosing action a when receiving observation o and following the policy π in the future. The *Bellman optimality equation* ensures that the optimal policy π^* provides the maximum value of $Q_{\pi}(b, a)$ in each possible condition, i.e.,

$$Q_{\pi^*}(b, a) = \max_{\pi} Q_{\pi}(b, a), \quad \forall (b, a) \in \mathcal{B} \times \mathcal{A}. \quad (3)$$

Solving the Bellman equation explicitly is only possible in very small problems, and this has led to the popularity of *Q-Learning* [35], which iteratively updates the Q-function as

$$Q^{(k+1)}(b(k), a(k)) = \alpha \left(r(k) + \lambda \max_a Q^{(k)}(b(k+1), a) \right) + (1 - \alpha) Q^{(k)}(b(k), a(k)), \quad (4)$$

where $\alpha \in [0, 1]$ is the so-called learning rate. Under most practical conditions, Q-Learning provably converges to the optimal policy if every observation-action pair is visited infinitely often. While convergence is not guaranteed, Deep Reinforcement Learning (DRL) algorithms, which use deep

neural networks to estimate Q-values, can deal with much more complex models and converge significantly faster. The *Double Q-Learning* algorithm is an extension of (4) with faster convergence [36], which can also be adapted to the DRL approach.

B. Cyber-Physical POMDPs

Let us consider a multi-agent scenario with a set of agents $\mathcal{I}$, which is the union of two subsets, named *sensors* ($\mathcal{G}$) and *robots* ($\mathcal{L}$): the sensors can observe the local state of the environment and transmit their observations, while the robots have more limited observations, but can act on the environment and change its physical state. Some agents might be able to act both as sensors and robots, i.e., may both communicate with other agents and act in the physical environment: in such cases, $\mathcal{G}$ and $\mathcal{L}$ are partially overlapping, i.e., $\mathcal{G} \cap \mathcal{L} \neq \emptyset$. However, each agent belongs to at least one of the subsets, and we have $\mathcal{G} \cup \mathcal{L} = \mathcal{I}$. The environment has a set $\mathcal{S}$ of possible states, and each agent $i \in \mathcal{G} \cup \mathcal{L}$ (either sensor or robot) receives observations from a set $\mathcal{O}_i$, also referred to as its *observation space*, depending on the current state. We denote the set of joint observations as $\mathcal{O} = \prod_{i \in \mathcal{G} \cup \mathcal{L}} \mathcal{O}_i$, such that a joint observation $o = (o_1, \dots, o_{|\mathcal{G}|+|\mathcal{L}|})$, with $o_i \in \mathcal{O}_i$, and the probability of having a given joint observation in each state as $P_o : \mathcal{S} \times \mathcal{O} \rightarrow [0, 1]$.

Hence, the state space consists of the union of two subsets: $\mathcal{S}_c$ and $\mathcal{S}_m$, which include *communication* and *control* states, respectively. If the environment is in a communication state, i.e., $s \in \mathcal{S}_c$, each sensor $g \in \mathcal{G}$ can transmit its information to other nodes, taking actions in the space $\mathcal{A}_g(s)$. This latter can be binary, with one action for transmission and one for silence, or have multiple values for different possible messages. On the other hand, if the environment is in a control state, i.e., $s \in \mathcal{S}_m$, each robot $\ell \in \mathcal{L}$ can interact with the physical environment, taking actions in the space $\mathcal{A}_\ell(s)$. Note that $\mathcal{S}_c \cup \mathcal{S}_m = \mathcal{S}$, i.e., at least one type of agent can make an action in each state. If $\mathcal{S}_c \cap \mathcal{S}_m \neq \emptyset$, there might be states that allow both agent types to act.

If the system is in state s , the joint agent actions are described by a vector $\mathbf{a} \in \mathcal{A}(s) = \prod_{g \in \mathcal{G}} \mathcal{A}_g(s) \times \prod_{\ell \in \mathcal{L}} \mathcal{A}_\ell(s)$, which is the concatenation of the single actions $a_i, \forall i \in \mathcal{G} \cup \mathcal{L}$. With a slight abuse of notation, we consider the actions taken by agents that are both sensors and robots twice. In particular, each agent may or may not act depending on the current state:

$$\mathcal{A}_g(s) = \{a_\emptyset\} \text{ if } s \notin \mathcal{S}_c, \quad \mathcal{A}_\ell(s) = \{a_\emptyset\} \text{ if } s \notin \mathcal{S}_m, \quad (5)$$

where $a_\emptyset$ is a null action with no effects. If an agent belongs to both subsets, its action is the combination of the (potentially null) communication and physical actions it selects. As in a classical MDP, the system evolution only depends on the current state and the robots' actions, and we denote the state transition probability function as $P_s : \mathcal{S} \times \mathcal{S} \times \mathcal{A}_\ell \rightarrow [0, 1]$.

In our model, there are two different reward functions: R_g (for the sensors) and R_ℓ (for the robots), which both have $\mathcal{S} \times \mathcal{A}$ as a domain and return values in $\mathbb{R}$. The robots' reward is not directly dependent on the sensors: given two different

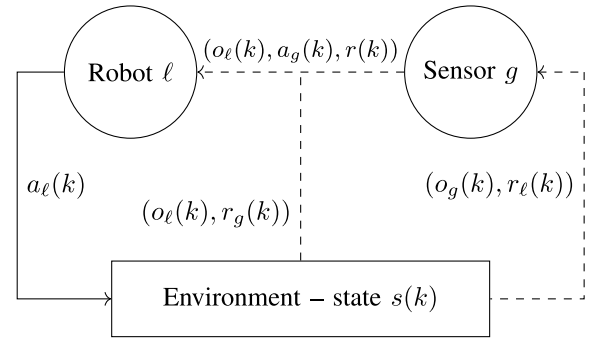


Fig. 1. Simple CP-POMDP model with two agents.

action vectors $\mathbf{a}, \mathbf{a}' \in \mathcal{A}$, $\mathbf{a} \neq \mathbf{a}'$, the robots' reward does not change if the robots' actions are identical in the two cases. Instead, the reward of the sensors depends on both their actions and those of the robots, as their goal is to aid the robots in their task in the physical environment. As we are considering a cooperative scenario, in which the goal of the sensors is to help the robots in their task, the communication performance entails two reward components. The first is the success of the communication, such as avoiding collisions or packet losses. The second is the *relevance* of the transmitted information, i.e., how much it helps the robots improve their performance.

A simple schematic of this model is given in Fig. 1, with one sensor and one robot: while the actions of the robot affect the environment, the sensor only affects the future observations of the robot, contributing to the task indirectly by giving the robot the most useful information. The full CP-POMDP model is then defined by the tuple $(\mathcal{G}, \mathcal{L}, \mathcal{S}_c, \mathcal{S}_m, \mathcal{A}_g, \mathcal{A}_\ell, P_s, P_o, R_g, R_\ell)$. In general, the nature of the model reflects the indirect effect of the sensors, whose actions do not affect the physical state of the environment, but only the information available to other nodes. They can then change the transition probabilities from a certain state to another that has the same physical environment but yields richer observations to all or a subset of robots. We highlight that, while VoI is often hand-designed, measuring a preset function of the estimation error made by the receiver (i.e., the robot) on the environment state, our formulation *implicitly* defines the value of transmissions through the reward function. Tautologically, a transmission that leads to a higher expected reward has a higher value, and the learned VoI can take into account future steps and the dynamics of the CP-POMDP, including the policy of the robot and sensors. The UoI metric defined in [12] plays a similar role, but including communications in the CP-POMDP formulation allows for more complex strategies than simple linear control, as well as solving a much wider class of problems.

By appropriately defining the state space and transition probabilities, it is possible to represent several different communication scenarios:

- *Scheduled transmission*: there is no interference between transmissions, but the maximum packet size might be too small to encode all the information each sensor has about the system state. The data will then need to be compressed, discarding part of the sensory observations

and deciding which parts of their observation are more important to transmit.

- *Slotted ALOHA*: the sensors need to coordinate to avoid interference. Each sensor can decide whether to transmit in a given state, or which resource block to use, and collisions significantly reduce the reception probability. This means that each sensor needs to estimate the usefulness of its information and make a decision on whether to transmit and risk a collision or just remain silent.
- *Spatial reuse*: the sensors have a limited range, so concurrent transmissions can be received if the interference is low enough. Naturally, this means that direct communication between a sensor and another agent outside of its range is impossible, and opens the possibility for multi-hop transmission.

In each scenario, only a limited number of packets, depending on the channel capacity, can be transmitted during the same communication state. The communication scheme can affect the performance of the strategies, and centralized approaches, with a full view of the environment, usually lead to better results. On the other hand, centralized transmission requires a significant signaling overhead, which may not be sustainable in resource-limited systems. In these cases, ALOHA-like access strategies might be the only available choice, and sensors must coordinate effectively even without a central scheduler. The spatial reuse case is even more complex, as coordination needs to take care of hidden terminal problems, further reducing performance if the sensors do not follow an explicit scheduling scheme.

Besides considering communication aspects, we can define specific scenarios depending on the number of robots and the effect of their actions:

- *Distributed perception*: there is a single robot (i.e., $|\mathcal{L}| = 1$) and all other nodes are sensors, whose objective is to augment the robot's awareness about the environment.
- *Coordination*: all nodes are both robots and sensors, and they must share their observations to achieve some form of consensus in order to perform an action.
- *Consensus*: this is a special case of coordination, in which actions can only be performed if a consensus threshold is met. A well-known example from the literature is the block-pushing task for a series of puck robots, who are too weak to push the block individually but can act in concert to accomplish the task [37].

C. Practical Considerations

The proposed CP-POMDP model can be used to represent a variety of cyber-physical systems in which several agents must coordinate to perform a task through a network. The main innovation of CP-POMDPs is the joint consideration of communication and control constraints, which only apply if the communication link between the agents actually represents a bottleneck. If the control process is extremely slow, giving agents the time to share their full observations, or the communication links have a very high capacity, enabling the agents to exchange those observations almost instantaneously,

the problem reduces to a traditional MARL scenario in which information on the state is shared by all agents.

In a generic CP-POMDP, the lower bound to the number of bits required for a sensor $g \in \mathcal{G}$ to communicate its observation is $H(o_g|h)$, where h is the history of past observations and $H(\cdot)$ represents the Shannon entropy function. Denoting by $C_{g,\ell}$ the capacity of the channel between g and a robot $\ell \in \mathcal{L}$, and by θ the period between two consecutive control steps, the condition for reducing the CP-POMDP to a simpler control problem is given by:

$$C_{g,\ell} \gg \frac{H(o_g|h)}{\theta}, \quad \forall g \in \mathcal{G}, \ell \in \mathcal{L}. \quad (6)$$

If the medium is shared, as in most wireless systems, we need to consider interference as well. Assuming that all nodes are in the range of each other, as in scheduled transmission or slotted ALOHA, the full perception of the system state is guaranteed by the following condition:

$$\sum_{g \in \mathcal{G}} \frac{H(o_g|h)}{\min_{\ell \in \mathcal{L}} C_{g,\ell}} \ll \theta. \quad (7)$$

From a practical perspective, CP-POMDPs are useful in practical scenarios where the channel capacity is insufficient for the agents to achieve a full perception of the system state in real time. A first class of use cases includes applications demanding a huge bandwidth since, e.g., the observations are very complex (i.e., $H(o_g|h)$ is large) or the control process is characterized by short deadlines (i.e., θ is small). This is the case of autonomous driving, in which agents make decisions on a scale of tens of milliseconds and their observations consist of images or depth maps recorded through cameras, radars, or LIDARs. Similar constraints apply to warehouses and manufacturing systems, in which robots must interact with human workers at a relatively high speed, optimizing operations while ensuring the workers' safety at all times.

The continuous sharing of multimedia data among vehicles and robots requires an extremely high throughput that may strain even 5G or future 6G technologies. In vehicular scenarios, selecting the right piece of information to forward to neighboring cars, while avoiding unnecessary transmissions, may provide a crucial upgrade of safety and efficiency. Information selection could take place through the semantic segmentation of the signals acquired by the on-board sensors, and the transmission of the sole elements that are useful for maintaining coordination between the different vehicles. Besides, in the case of a congested channel, it is necessary to prioritize communication for the vehicles whose state is changing very fast and unpredictably because, e.g., of an abrupt emergency maneuver. In such a context, the use of CP-POMDPs can allow the vehicles to coordinate and, consequently, minimize the risk of car accidents.

A somewhat complementary category of use cases involves applications constrained by limited communication channels (i.e., $C_{g,\ell}$ is small). These limitations may arise from factors such as low capacity, high propagation delay, or a combination of both. Prominent examples include wide-area drone swarms, which may employ long-range communication links with low bitrate to connect dispersed nodes [22], and underwater

networks, where acoustic waves take the place of radio-frequency waves, resulting in significantly reduced bandwidth and slower propagation speeds. The extremely low channel capacity and, in some cases, prolonged signal propagation delays, pose significant challenges in achieving cooperative perception among multiple agents. This scenario underscores the necessity to develop strategies tailored for situations where various agents, such as drones or underwater sensors, only have access to fragmented information about their environment and have to exploit at best the limited transmission capacity of their communication channels. The capability of CP-POMDPs to account for the cost of unnecessary transmissions is hence fundamental to model scenarios in which bandwidth is a scarce resource.

Besides, we observe that CP-POMDPs can be further extended by considering non-cooperative settings, in which agents need to balance selfish objectives with the system-level reward. For example, this is the case of applications where robots or drones are reluctant to deplete their limited batteries. The complexity of non-cooperative scenarios is significantly higher since withholding or even falsifying information about the environment may be beneficial to individual agents, and the problem of trust and reliability becomes central. Analyzing non-cooperative systems goes beyond the scope of this paper and is left for future work.

IV. DISTRIBUTED PERCEPTION MODEL

In this section, we define a model for a distributed perception scenario, with a single robot, aided by several sensors. The model will be then evaluated in two use cases, presented in the later sections of the paper.

A. System Model

We consider a grid world scenario, in which the map is represented by a square grid of size $N \times N$. Each cell of the map is identified by a coordinate vector $\mathbf{x} \in \mathcal{M} = \mathcal{N} \times \mathcal{N}$, where $\mathcal{N} = \{1, \dots, N\}$. The single robot, whose position in slot k is denoted by $\mathbf{x}_\ell(k)$, must reach a set $\mathcal{T}$ of mobile targets: the position of the i -th target in slot k is denoted by $\mathbf{t}_i(k)$. At the same time, the agent must avoid moving obstacles in the scenario. The final goal is to reach a known final position $\mathbf{x}_f$ after visiting all targets. To achieve this goal, the perception of the robot is augmented by a set of sensors $\mathcal{G}$, which can perceive their surrounding environment and communicate their observations to the robot.

Let us denote the robot's position at slot k by $\mathbf{x}_\ell(k)$. Following a common RL approach, we divide the time into episodes, each of which lasts $K \in \mathbb{N}$ slots. We also define the environment matrix $\mathbf{M}(k) \in \{\phi, \omega, \tau\}^{N \times N}$ over the set $\mathcal{M}$ of possible cell coordinates $\mathbf{x}$, whose elements are equal to ϕ if the corresponding cell is free, ω if it contains an obstacle, and τ if it contains a target. We highlight that the environment matrix is dependent on time, as obstacles and targets may move during the course of an episode.

We denote the robot's belief on the state of the environment as $\hat{\mathbf{M}}_\ell(k) \in \{\nu, \phi, \omega, \tau\}^{N \times N}$: value ν indicates that the robot has no knowledge of the content of a cell. The robot starts

with an empty belief matrix $\hat{\mathbf{M}}_\ell(0, \mathbf{x}) = \nu, \forall \mathbf{x} \in \mathcal{M}$. At the beginning of each slot k , it can directly sense the status of cells within a limited field of view $\mathcal{V}_\ell(k)$, defined as the set of cells whose Euclidean distance from the robot's current position is lower than a certain value v_ℓ , and update those values in $\hat{\mathbf{M}}_\ell(k)$. The belief matrix is then updated:

$$\begin{aligned} \hat{\mathbf{M}}_\ell(k, \mathbf{x}) = & M(k, \mathbf{x})I(\|\mathbf{x} - \mathbf{x}_\ell(k)\|_2 \leq v_\ell) \\ & + \hat{\mathbf{M}}_\ell(k-1, \mathbf{x})I(\|\mathbf{x} - \mathbf{x}_\ell(k)\|_2 > v_\ell), \end{aligned} \quad (8)$$

where $\|\cdot\|_2$ denotes the Euclidean norm and $I(x)$ denotes the indicator function, equal to 1 if x is true and 0 otherwise. We also include a matrix $\Delta_\ell(k) \in \mathbb{N}^{N \times N}$ in the robot's observation, which represents the AoI of each cell, i.e., how long ago each cell was last updated:

$$\Delta_\ell(k, \mathbf{x}) = (\Delta_\ell(k-1, \mathbf{x}) + 1)I(\|\mathbf{x} - \mathbf{x}_\ell(k)\|_2 > v_\ell), \quad (9)$$

with $\Delta_\ell(0, \mathbf{x}) = \infty, \forall \mathbf{x} \in \mathcal{M}$. The robot's full observation at step k is then the tuple $o_\ell(k) = (\mathbf{x}_\ell(k), \hat{\mathbf{M}}_\ell(k), \Delta_\ell(k))$. Correspondingly, the robot's observation space, i.e., the set that this observation belongs to, is $\mathcal{O}_\ell = \mathcal{M} \times \{\nu, \phi, \omega, \tau\}^{N \times N} \times \mathbb{N}^{N \times N}$.

The robot's actions are simple, as it can move either horizontally or vertically by one cell. Hence, its action space is given by 4 possible movements: $\mathcal{A}_\ell = \{(1, 0), (-1, 0), (0, 1), (0, -1)\}$, corresponding to a step to the left, right, up, and down, respectively. However, the robot can never cross the map boundaries, and obstacles are impenetrable, so if the robot tries to run into an obstacle or exit the map, its action will have no effect. If the robot performs action $a_\ell(k) \in \mathcal{A}_\ell$ at slot k , its position in the next slot is given by:

$$\begin{aligned} \mathbf{x}_\ell(k+1) = & \mathbf{x}_\ell(k) + a_\ell(k)I(\mathbf{x}_\ell(k) + a_\ell(k) \in \mathcal{M}) \\ & \times I(M(k, \mathbf{x}_\ell(k) + a_\ell(k)) \neq \omega). \end{aligned} \quad (10)$$

The system is always in a control state (i.e., $\mathcal{S}_m = \mathcal{S}$), which implies that the robot performs an action $a_\ell(k)$ at each slot k .

Regarding sensors, the communication channel is severely constrained, such that the system is in a communication state $s \in \mathcal{S}_c$ only when $\text{mod}(k, K_p) = 0$, where $\text{mod}(\cdot)$ is the integer modulo function; that is, sensor actions are separated by K_p slots. Besides, only one packet can be transmitted in each communication slot, i.e., only one sensor can communicate its observation in a given slot, as multiple simultaneous transmissions will result in a destructive collision. As all slots are control slots, both the sensors and the robot will act at the same time: the sensors will transmit their information, while the robot will move.

The sensors also have a limited field of view, with a maximum distance v_g , while the position $\mathbf{x}_g$ of each sensor g is fixed. We define sensor g 's field of view $\mathcal{V}_g$:

$$\mathcal{V}_g = \{\mathbf{x} \in \mathcal{M} : \|\mathbf{x} - \mathbf{x}_g\|_2 \leq v_g\}. \quad (11)$$

At each communication slot k , the sensor g can transmit information over a subset $\mathcal{C}_g(k) \subseteq \mathcal{V}_g$ of its field of view, corresponding to a limited area of the map. After a successful communication, the robot's belief matrix $\hat{\mathbf{M}}'_\ell(k, \mathbf{x})$ is updated as:

$$\hat{\mathbf{M}}'_\ell(k, \mathbf{x}) = M(k, \mathbf{x})I(\mathbf{x} \in \mathcal{C}_g(k)) + \hat{\mathbf{M}}_\ell(k, \mathbf{x})I(\mathbf{x} \notin \mathcal{C}_g(k)). \quad (12)$$

The value of $\Delta'_\ell(k)$ is also reset to 0 for all cells in the transmitted area $\mathcal{C}_g(k)$.

We consider two communication settings, corresponding to the scheduled and slotted ALOHA setups, defined in Section III-B:

- In the centralized control scenario, a single sensor g (with $|\mathcal{G}| = 1$) can perceive the full map, i.e., $v_g = \infty$ and $\mathcal{V}_g = \mathcal{M}$. In this case, the channel capacity is the only constraining factor and, at each communication slot k , the sensor policy should establish the area $\mathcal{C}_g(k) \in \mathcal{C}$ to be transmitted, where $\mathcal{C}$ is a partition of the map. In particular, the sensor action space is defined as $\mathcal{A}_G = \mathcal{C}$. This configuration is easy to manage, as communication decisions are centralized and communication is always successful.
- In the distributed control scenario, there are multiple sensors, i.e., $|\mathcal{G}| > 1$, that can transmit over a shared collision channel. If the system is in a communication state, each sensor can independently decide whether to transmit information about its field of view (action 1) or remain silent (action 0); therefore, each sensor's action space is $\mathcal{A}_G = \{0, 1\}$. If at the slot k the sensor g chooses action 1, we have that $\mathcal{C}_g(k) = \mathcal{V}_g$. However, since the transmission is over a collision channel, the transmission is only successful if $\sum_{g \in \mathcal{G}} a_g = 1$. Since the sensors act simultaneously without any explicit coordination, it is challenging to find a policy that allows the robot to receive enough information while minimizing the risk of packet collisions. In particular, each sensor is not aware of the information observed by the others and needs to consider whether its observations are important enough to risk a collision or whether some other sensor may have more important information. These issues are inherent in random access models such as slotted ALOHA.

The ultimate goal for the robot is to accomplish its task in as few steps as possible; as such, it will get a large reward ρ when it collects all the mobile targets and reaches the target position within a predetermined time, while no reward is given if the episode ends before it has managed to do so. Besides, to accelerate the training process, we introduce an intermediate reward σ whenever specific conditions are met. At each slot k , the reward $r_\ell(k)$ for the robot is:

$$r_\ell(k) = \rho I(\mathbf{x}_\ell(k) = \mathbf{x}_f) I(\forall t \in \mathcal{T}, \exists j : \mathbf{x}_\ell(j) = \mathbf{x}_t(j)) + \sigma I(\chi(s(k), a_\ell(k)) = 1), \quad (13)$$

where $I(\cdot)$ is the indicator function (as defined in Section IV-A), while $\chi(s(k), a_\ell)$ returns 1 whenever the robot gets closer to the task completion. The full definition of $\chi(s(k), a_\ell)$ is reported in the Appendix. The first term on the right-hand-side of (13) is always equal to zero except when the robot is in the target position ($\mathbf{x}_\ell(k) = \mathbf{x}_f$) and all the targets in $\mathcal{T}$ have been collected ($\forall t \in \mathcal{T}, \exists j : \mathbf{x}_\ell(j) = \mathbf{x}_t(j)$). The second term on the right-hand-side of (13) is always equal to zero except when the conditions expressed by $\chi(s(k), a_\ell)$ are met.

The reward for each sensor g is given by the reward for the robot over the next time horizon, with an additional penalty η

for the possible packet collisions:

$$r_g(k) = \sum_{j=k}^{k+K_p} r_\ell(j) - \eta I(a_g(k) \neq 0) I\left(\sum_{j \in \mathcal{G} \setminus \{g\}} a_j(k) \geq 1\right). \quad (14)$$

In particular, the second term on the right-hand-side of (14) is always equal to zero except when the sensor g transmits ($a_g(k) \neq 0$), causing a collision because another sensor is also transmitting, i.e., we have $\sum_{j \in \mathcal{G} \setminus \{g\}} a_j(k) \geq 1$. We observe that there is no penalty in the centralized case, as the sensor does not suffer from any interference and communication is ideal. Hence, only VoI is considered for scheduling transmissions, as the reward of the centralized scheduler is the same as the one of the robot. On the other hand, the reward in the distributed case balances the VoI with medium access considerations, reducing the risk of collisions by limiting transmissions.

B. Learning Architecture

To optimize our system, we exploit a DRL approach, where the policy of each agent in $\mathcal{I}$ is implemented by a DQN, i.e., a Neural Network (NN) architecture approximating the Q-function $Q(\cdot)$ [6]. In the centralized control scenario, we define the functions $Q_\ell(\cdot)$ and $Q_g(\cdot)$, which determine the robot's control strategy and the sensor's communication strategy, respectively. Instead, in the distributed control scenario, we have a total of $1 + |\mathcal{G}|$ Q-functions, i.e., one managing the robot, and an additional function for each of the sensors.

In the case of the robot, $Q_\ell(\cdot)$ is approximated by a Convolutional Neural Network (CNN) with 4 output neurons, i.e., one for each possible control action. Instead, the CNN input is a convolutional layer that handles the robot's observation as an $N \times N$ image with 4 channels. The input channels are given by the matrices $\hat{\mathbf{M}}_\ell(k)$, and $\Delta_\ell(k)$, as well as the one-hot encoding matrices $\mathbf{X}_\ell(k)$ and $\mathbf{X}_f$, which represent the encodings of $\mathbf{x}_\ell(k)$ and $\mathbf{x}_f$, respectively. Therefore, at each slot k , the neural network takes these 4 matrices as its input information and returns the expected cumulative reward associated with each action $a \in \mathcal{A}_\mathcal{L}$ as outputs.

The function $Q_g(\cdot)$ of each sensor $g \in \mathcal{G}$ is instead approximated by a CNN using a convolutional layer with 5 channels as input, and a linear layer with $|\mathcal{A}_G|$ neurons as output. In this case, each observation is treated as an $N \times N$ image with 5 channels: the first 4 channels encode the same information as the robot, while the fifth encodes the matrix $\hat{\mathbf{M}}_{g,k}$, which represents the sensor's knowledge over the system state:

$$\hat{\mathbf{M}}_{g,k}(\mathbf{x}) = M(k, \mathbf{x}) I(\mathbf{x} \in \mathcal{V}_g) + \nu I(\mathbf{x} \notin \mathcal{V}_g). \quad (15)$$

As for the robot, the number of output neurons is equal to the cardinality of the action space, which is 2 in the distributed scenario, and the number of sensors in the centralized scenario.

The hidden layers of all the learning architectures (for both the robot and the sensors) include two convolutional layers, which transform the 2-dimensional input into a vector to be fed to the output layer. We consider the Rectified Linear Unit

TABLE I
AGENT ARCHITECTURE

Agent	Layer							
	Input (convolutional)		Hidden (convolutional)		Hidden (convolutional)		Output (linear)	
Robot	Input channel	4	Input channel	64	Input channel	64	Input neurons	64
	Output channel	64	Output channel	64	Output channel	64		
	Kernel	(3,3)	Kernel	(3,3)	Kernel	(3,3)	Output neurons	4
	Stride	1	Stride	(4,4)	Stride	(4,4)		
Sensor	Input channel	4	Input channel	64	Input channel	64	Input neurons	64
	Output channel	64	Output channel	64	Output channel	64		
	Kernel	(3,3)	Kernel	(3,3)	Kernel	(3,3)	Output neurons	{2, 9}
	Stride	1	Stride	(4,4)	Stride	(4,4)		

(ReLU) activation function for the hidden layers because of its efficiency in representing non-linear dynamics, while we used a linear function in the output layer. More details about the learning architectures are reported in Tab. I.

To achieve a Joint Communication and Control (JCC) optimization, in which the robot and sensors' policies are optimized for each other, we consider an iterative approach for training the system. This is a common practice in MARL, as multiple agents exploring the environment and improving their policies at the same time can prevent convergence [38], [39]. Instead, by only training one kind of agent at a time, we ensure that the environment that the agent sees is Markovian. Specifically, we organize the training into N_{round} rounds, each of which includes two phases: the first lasts N_{train}^{ℓ} episodes and is dedicated to the robot training; the second lasts N_{train}^g episodes, and is dedicated to the sensor training.

In the first phase of each round, the policy of the sensors is fixed (in the first round, the sensors are initialized with a pre-determined strategy), while the robot's DQN is re-initialized and trained to adapt to the current communication strategy. In the second phase of each round, the configuration is flipped: the robot's policy is now fixed, while the sensors' DQNs are re-initialized and trained to adapt to the robot's policy. Re-initializing the sensors' and the robot's policy at the beginning of each phase ensures that the agents' greedy actions are not biased toward previous solutions, thus enabling a wider exploration of the learning environment. Once this iterative training phase is over, we train the robot architecture again for N_{train}^{ℓ} episodes. Hence, the total number of robot training episodes is $N_{\text{train}}^{\ell} \times (N_{\text{round}} + 1)$, while the sensors are trained for $N_{\text{train}}^g \times N_{\text{round}}$ episodes.

C. Game Theoretical Considerations

As in any MARL scenario, we can see the CP-POMDP model as a Markov game [40] between multiple agents. The iterative training framework described in Section IV-B is a version of Iterated Best Response (IBR) in the space of the agent policies, as each agent optimizes its response to the other's frozen policy. In a general Markov game, IBR has no convergence guarantees, as the iterative procedure may form loops. On the other hand, the distributed perception scenario presented in this section represents a *potential game* [41], where all the agents seek the maximization of the reward $R_{\mathcal{L}}(\cdot)$ of the robots. Hence, our model is not affected by the

same non-stationarity issues as *non-cooperative* or other types of games, and IBR always converges to a Nash Equilibrium (NE) [40, Th. 6]. At the same time, even potential games may present multiple NEs, and the solution obtained via IBR is not guaranteed to be Pareto optimal [42].

The scheduled communication scenario involves a single sensing agent deciding on which piece of information to transmit and represents a well-studied problem in the literature [10]. In this case, the Markov game involves two agents only but there still may be multiple NEs [43], only one of which leads to the optimal performance. For instance, the solution maximizing the cumulative reward could be very expensive from a communication point of view, while sub-optimal solutions may require less frequent updates and, consequently, seem more attractive. To mitigate this issue, a possible strategy is to perform multiple training runs with different initial policies for the sensors and robots.

If we consider a collision-prone communication channel (e.g., slotted ALOHA), simultaneous transmissions from multiple sensors can lead to packet collisions and losses, making it more difficult to achieve convergence. In this case, the above-mentioned theoretical guarantees [40, Th. 6] still hold if the reward is the same for all agents, but the number of NEs is potentially much higher. In particular, any deterministic outcome with a single sensor transmitting in every communication state represents a possible NE for the system. In such a case, any deviation from the equilibrium will lead to a packet collision or a silent slot: therefore, none of the sensors has the incentive to deviate unilaterally from the equilibrium. Reaching a near-optimal NE, in which communication is optimized jointly with control, may be theoretically intractable in the general case. Finding the best NE in a Markov game is a PPAD-hard problem [44], which is computationally challenging. Practical approaches to reach a near-optimal NE, without relying on a IBR optimization, might be achieved by equipping agents with explicit models of the system state, as well as using Shapley value theory [45] to tune the global reward function.

V. SIMULATION SETTINGS AND RESULTS

In this section, we implement our CP-POMDP model in two underwater use cases, which reflect the distributed perception model from Section IV. In these tasks, an AUV (which acts as the single robot in our model) must reach a surface vessel

on the other side of a map, while either avoiding underwater debris or visiting a set of nodes to recover data. Multiple buoys act as sensors; in the centralized case, the buoys also have wireless connections over the air, and so can act as a single entity and share information, while in the distributed case, they must use a shared underwater acoustic channel and risk packet collisions.

In the following, we first describe the benchmark strategies against which our learning framework is tested and present the simulation settings. Hence, we analyze the system performance in the two test scenarios, highlighting the benefits and drawbacks of the proposed JCC approach.

A. Benchmark Strategies

As benchmarks for our model, we consider three learning systems where only the AUV is controlled by a learning agent. In such a case, the buoys follow a pre-determined communication strategy, while the AUV adapts its control actions to the transmission process, considering the buoys as part of the learning environment. This makes the learning environment more stable but constrains the final performance by the pre-determined communication strategy. In practice, at each communication slot, the buoy's transmissions are managed as follows.

- *Random Communication (RC)*: at each communication slot k , the buoys transmit a random portion of the map $C_g(k) \in \mathcal{C}$.
- *Closest Communication (CC)*: at each communication slot k , the buoys transmit the map sub-area $C_g(k) \in \mathcal{C}$ whose center is closest to the AUV's coordinates $\mathbf{x}_\ell(k)$.
- *Oracle Communication (OC)*: at each slot k , the AUV receives an update of the entire map, so that $\hat{M}_\ell(k) = M(k) \forall k$.

We observe that the OC strategy represents an upper bound for the system performance, as it models a case in which there is no communication bottleneck and the AUV is always aware of the real map state. Instead, the RC and CC strategies consider realistic communication and can be implemented in a distributed fashion by pre-programming the buoys with no collisions or loss of performance. To provide an additional comparison tool, we also consider the following learning-based strategy [13]:

- *SchedNet*: the communication system considers a centralized transmission model and consists of a single NN architecture, whose output layer includes a neuron for each buoy. Each output of the NN is the prediction of the Q-value of the AUV's DQN in the case of transmission. The SchedNet architecture is trained concurrently with the AUV and, after reaching convergence, the buoy associated with the higher Q-value (i.e., the higher VoI) will be the one to transmit. The concept of SchedNet is inspired by the same principle as our JCC strategies, but the limited nature of the scheduler, which is not a true DRL agent, prevents it from adjusting the VoI to its own policy as well as the AUV's.

We remark that SchedNet can be implemented in a distributed fashion, as in the original work, only if the medium access

scheme supports an underlying conflict resolution mechanism, i.e., if the nodes use some form of reservation mechanism or Carrier Sense Multiple Access (CSMA), as the system assumes a collision-free channel.

All the above strategies are tested against two configurations of our JCC framework. The first, named Centralized JCC (CJCC), is based on the centralized control system defined in Section IV-A, while the other, named Distributed JCC (DJCC), associates each buoy with a different agent, as in the distributed control scenario given in the same section. From a communication perspective, CJCC and SchedNet both implement a scheduled transmission system, as a central entity orchestrates the transmission process and there are no packet collisions. In these schemes, the buoys have a full view of the environment and, thus, the problem is to determine what part of the system state is more valuable for the AUV. Instead, the DJCC approach corresponds to the slotted ALOHA system, which makes it more challenging for the AUV to perceive the environment, as packet collisions are possible. In particular, each buoy observes a different portion of the state, requiring each agent to implicitly estimate the knowledge of the other agents while tuning the communication policy.

B. Underwater Tasks and Scenario Settings

We define the two underwater tasks for our performance evaluation. In both cases, we consider the reference model described in Section IV, with a map of size $N = 12$, divided into $T = 9$ different non-overlapping portions. In the centralized control scenario (used by all the benchmarks and the CJCC system), there is a single sensor with 9 possible actions, i.e., one action for each buoy. Instead, DJCC considers 9 different sensors, each of which can perceive and transmit a specific portion of the map. We set the period between two consecutive communication states to $K_p = 5$ slots and the maximum episode duration to $K = 100$ slots. In our scenario, the communication bottleneck is given by the transmission frequency: the buoys can only transmit one packet every K_p slots and, therefore, need to prioritize information that is useful for the AUV. This type of restriction fits an underwater acoustic channel, which has long propagation times due to the limited speed of sound and correspondingly long guard intervals between transmissions [46].

We fix the maximum episode duration to $K = 100$ slots: if the AUV spends more time in the map without finishing the task, it fails to achieve its goal. If the AUV reaches the surface vessel with the data, it receives a bonus $\rho = 10$, while the collision penalty and intermediate reward parameters are set to $\eta = 1$ and $\sigma = 0.22$, respectively. More details about the mathematical definitions of the underwater tasks are given in the Appendix.

When training the benchmark strategies, we consider a single training phase. On the other hand, the training of the JCC systems includes $N_{\text{round}} = 2$ rounds, where the AUV training phases last $N_{\text{train}}^g = 100000$ episodes each, and those of the buoys $N_{\text{train}}^g = 500000$ episodes each. We observe that N_{train}^g is higher than N_{train}^ℓ , since the buoys take actions less frequently than the AUV and, consequently,

TABLE II
SIMULATION SETTINGS

Parameter	Value	Description	Parameter	Value	Description
$ \mathcal{L} $	1	Number of AUVs	$ \mathcal{G} $	9	Number of buoys
$N \times N$	12×12	Map size	λ	0.95	Discount factor
$ \mathcal{C} $	9	Number of map partitions	ζ	10^{-5}	Learning rate
K	100	Steps per episode	N_{round}	2	Training rounds
K_p	5	Inter-transmission period	N_{train}^{ℓ}	100000	Training episodes for the AUV
ρ	1.0	Final reward	N_{train}^g	500000	Training episodes for the buoys
σ	0.22	Intermediate reward	N_{test}	10000	Testing episodes

need more episodes to reach convergence. During a single training round, the agents use an ε -greedy policy to explore the observation-action space. The policy selects the best possible action with probability $1 - \varepsilon$ and a random suboptimal action with probability ε , with the value of ε slowly decreasing from 0.9 to 0.1. Finally, we set the discount factor to $\lambda = 0.95$ and we exploit the Adaptive moment estimator (Adam) algorithm to optimize the NN training [47], considering $\zeta = 0.00001$ as the maximum learning rate. All the simulation parameters are reported in Table II.

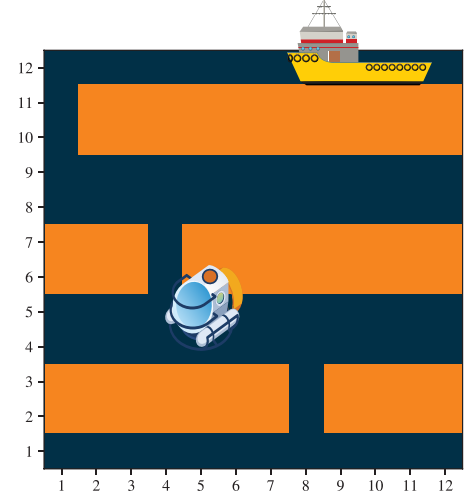
In both tasks, the final position corresponds to the position of a surface vessel on the top row of the map, which is fixed for the duration of an episode but may be different for each episode. The initial position of the AUV is also randomly generated on the bottom row of the map. Fig. 2 gives a visual representation of the two tasks, which are briefly described below.

1) *Debris Avoidance*: In this scenario, there are several obstacles in the marine environment, such as rocks, sea mines, or oil installations, and the AUV must reach the vessel as fast as possible without crashing into those obstacles. In general, the debris locations change in time (accounting for currents and drift), so they correspond to the obstacles in $\mathbf{M}(k)$. Besides, there are no targets in this environment, i.e., $\mathcal{T} = \emptyset$. In the simulations, we model the obstacles as 3 horizontal walls, each of which has a single opening that may move over time. The intermediate reward is given to the AUV if it moves upward in the map, or if it moves sideways toward an opening when the way upward is blocked.

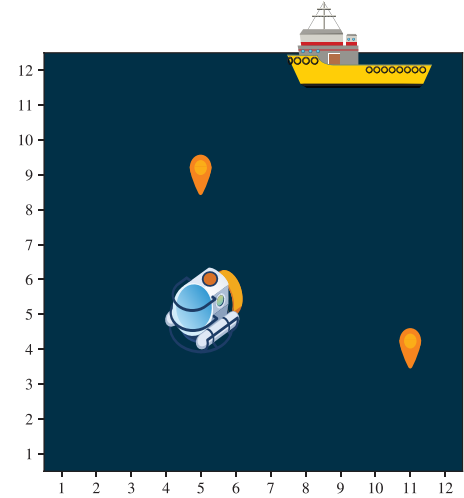
2) *Data Muling*: In data muling, the AUV has to visit a set of underwater floating nodes to recover their data through short-range communications [48] before reaching the vessel to upload the data. We consider a simple case with only 2 targets, i.e., $|\mathcal{T}| = 2$. Particularly, in this task, we do not consider any obstacles, i.e., $\nexists \mathbf{x} \in \mathcal{M} : M(k, \mathbf{x}) = \omega, \forall k \in \mathbb{N}$.

C. Results

As the main performance metric of our experiments, we consider the number N_{step} of steps that the AUV takes to perform its task. Particularly, if the AUV is not able to achieve its goal before the episode ends, we have that $N_{\text{step}} = K$, which is the maximum episode duration. Fig. 3 shows the mean of N_{step} , as well as the 25th and 75th percentiles of its distribution, during the training of the MARL strategies in both scenarios; we show only the DJCC strategy for the debris avoidance scenario, as the two have extremely similar



(a) Debris avoidance.



(b) Data muling.

Fig. 2. Depiction of the two underwater tasks.

performance, and only the CJCC strategy in the data muling scenario, as it is the best of the two. In the beginning, both the 25th and 75th percentiles of N_{step} remain equal to 100 (i.e., the total number of steps in an episode) since the AUV is not capable of completing its mission in more than 75% of the episodes, regardless of the strategy adopted during the training. Instead, as the training goes on, both the quartiles and the expectation of N_{step} decrease toward the minimum value.

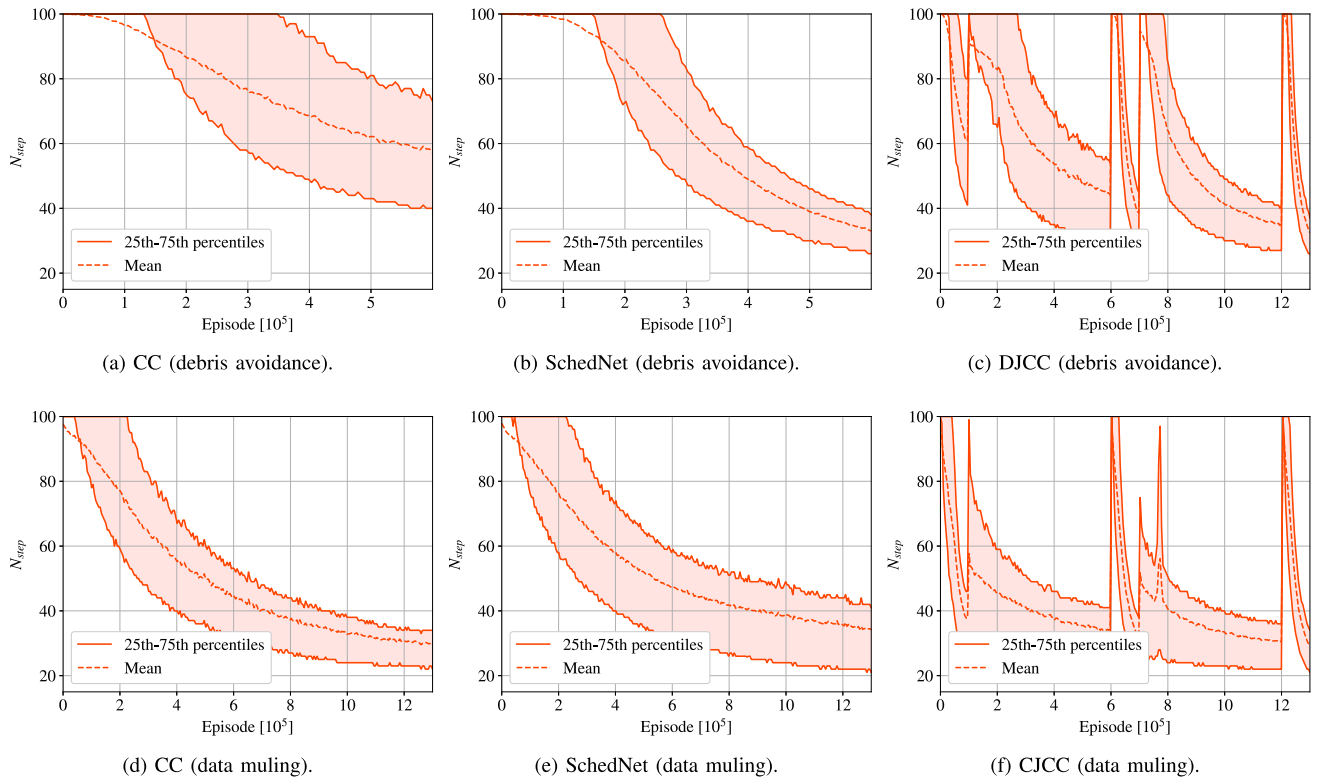


Fig. 3. Training performance and convergence of the MARL schemes.

In the case of SchedNet and CC, the performance improves monotonically, while, by using the JCC strategies, it exhibits a periodic pattern, where the average of N_{step} repeatedly drops from 100 to about half of that value. This is because the NNs are reset after each round of the iterative training, making the AUV and the buoys forget the policy learned in the past. At the same time, as more training rounds are completed, the average N_{step} becomes significantly lower since the agents have progressively learned to adapt to each other.

In general, we consider a total of 1.3×10^6 training episodes for each approach but the SchedNet and CC strategies in the debris avoidance scenario. These schemes are indeed characterized by a simpler training process and require only 6×10^5 episodes to converge to a stable solution. The convergence was assessed by monitoring the hierarchy between the different Q-values during the training, which stabilizes after approximately 2×10^5 episodes. We recall that the system convergence cannot be deduced by Fig. 3 because of the ϵ -greedy exploration policy. During the training, performance is affected by both greedy and random actions, so that a reduction of the mean N_{step} is not necessarily associated with a policy improvement.

The same figure highlights how the training of the JCC strategies is organized into rounds, unlike the benchmark approaches, so that each round is dedicated to the training of either the buoys or the AUV. In particular, using the JCC schemes, the nodes are trained for different numbers of steps: the AUV is only trained for 3×10^5 episodes, while the remaining episodes are used for the buoys. For evaluating the final performance, we carry out a total $N_{\text{test}} = 10000$ testing episodes for each strategy. The distribution of N_{step} during

the test phase for the two tasks is presented as a boxplot in Fig. 4. The whiskers are the 5-th and 95-th percentiles, while the edges are the 25-th and 75-th, and the white line in the middle of each box is the median.

As expected, OC outperforms all other strategies, with the lowest values of N_{step} . In the debris avoidance scenario, the AUV accomplishes its goal in less than 30 steps in half of the testing episodes, while the distribution median decreases to 22 steps in the data muling case. This is because the OC scheme allows the AUV to perfectly know the environment, making the problem trivial to solve. On the opposite extreme, the RC strategy leads to the worst results, as the sensor decisions do not consider the relevance of the information to the AUV when choosing what to transmit. This leads to a median completion time of 39 steps in data muling, while the AUV fails to reach the vessel in more than 50% of the test episodes in the other scenario. The CC scheme has an intermediate performance, showing a median of 57 and 32 steps to complete an episode in the debris avoidance and data muling scenario, respectively.

The RC and CC strategies are significantly outperformed by the JCC schemes, which can almost get the same performance as the ideal communication setup. In particular, both DJCC and CJCC can achieve superior performance in the debris avoidance mission, coordinating communications so as to always deliver the most relevant information to the AUV. SchedNet also performs very well in this scenario, matching the performance of the JCC strategies. Indeed, the optimal policy is relatively easy and SchedNet manages to converge to it easily. The data muling scenario is more difficult, and even more so for a distributed setup: as buoys often only know

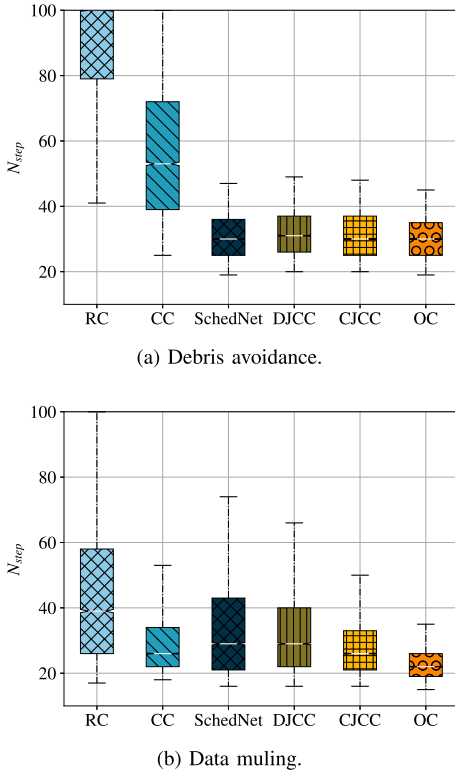


Fig. 4. Performance of the different strategies for the two tasks.

the position of one of the two nodes, the risk of collisions is high, and the buoys will tend to behave more conservatively. In this case, the median value of N_{step} is 26 and 30 for CJCC and DJCC, respectively, which is still significantly lower than the RC and CC strategies. On the other hand, SchedNet shows the same median performance as DJCC, despite the lack of collisions: it then represents a centralized approach that is less flexible and more expensive than the one offered by DJCC. The additional flexibility obtained by CJCC by making the buoys full-fledged agents in the CP-POMDP and training iteratively does not pay back in the simpler debris avoidance scenario, where SchedNet performs equally well, but becomes crucial in this more complex task. Indeed, in the data muling case, the CC strategy, which avoids collision by design, performs better than SchedNet and DJCC, even though slightly worse than CJCC.

A possible reason for the better performance of iterative training is that the joint training of both scheduling and AUV, as done by SchedNet, makes the environment non-Markovian, making the training much less stable: as both agents are updating their policy at the same time, the environment for each one will change as the other's policy is updated, causing significant convergence issues. This is even more noticeable when looking at the worst-case performance, represented by the upper whiskers of the boxplot: the difference between the schemes is extremely stark, showing the benefits of the JCC schemes over all the benchmarks. Hence, the CJCC solution, which combines the advantages of JCC with a centralized communication process, is the closest to the ideal scenario (represented by the OC strategy) and can deliver consistently

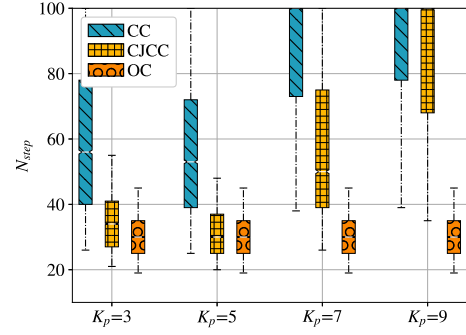


Fig. 5. Performance of the CC, CJCC, and OC strategy while varying K_p (debris avoidance).

good performance in both scenarios, while the other tested policies fail in one or the other.

We also consider performance as a function of the inter-transmission period K_p to analyze the adaptability of the proposed approach to a tighter bottleneck in the communication link. Fig. 5 shows a boxplot of the duration of the debris avoidance mission for the ideal, CC and CJCC approaches for $K_p \in \{3, 5, 7, 9\}$. Naturally, the performance of the OC scheme is invariant to the value of K_p , as the AUV has always a full perception of the map. Interestingly, considering $K_p = 3$ makes the performance slightly worse for both the CC and CJCC schemes. This is because the debris moves once every 5 steps, and transmissions are perfectly synchronized with the evolution of the map: if the buoys transmit the right information, the AUV will immediately know where to go. Introducing an offset between changes in the environment and transmissions would make performance worse, perhaps leading to more frequent transmissions to be more convenient. On the other hand, increasing the value of K_p drastically degrades performance. Considering the CJCC strategy, the median value of N_{step} goes from 30 to 50 when $K_p = 7$, while the AUV fails to reach the vessel in more than 50% of the episodes when $K_p = 9$. The CC strategy performs even worse, failing more than half of the test episodes even with $K_p = 7$.

To better illustrate the agents' behavior, Fig. 6(a) shows the distribution of the AUV position in the obstacle avoidance scenario with different control strategies. The heatmaps do not refer to any single realization of the scenario, but rather to the average over the 10000 testing episodes, each of which has an independent random initial obstacle position. The distribution is presented as a colormap: the lighter the cell color, the higher the probability of the AUV to be in the corresponding position during the testing phase. We can notice that the AUV tends to keep close to the center of the map when using the CC scheme, never straying too far to the left or right: on the other hand, SchedNet and the JCC strategies have a distribution more similar to the OC case. This is due to the influence of transmissions from the buoys: if the AUV can know where the opening is, it can move directly toward it, instead of staying close to the center of the map and then gradually exploring. While this is a relatively simple example, and constructing a better transmission heuristic might be possible, the joint design

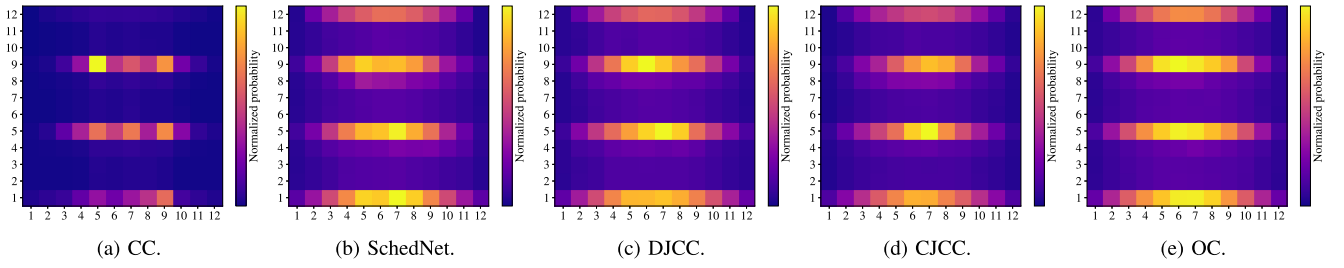


Fig. 6. Heatmap of the AUV location probability in the debris avoidance task.

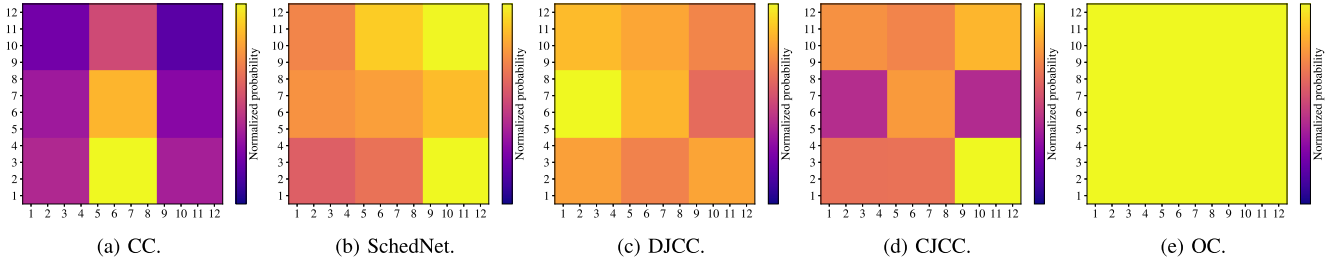


Fig. 7. Heatmap of the transmitted area probability in the debris avoidance task.

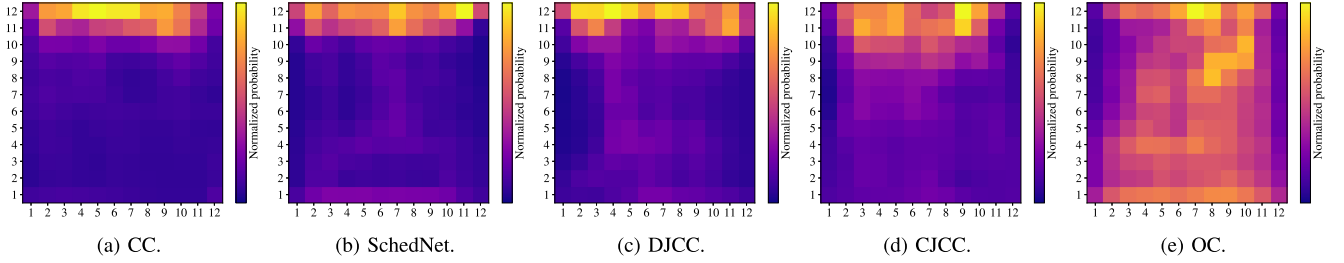


Fig. 8. Heatmap of the AUV location probability in the data muling task.

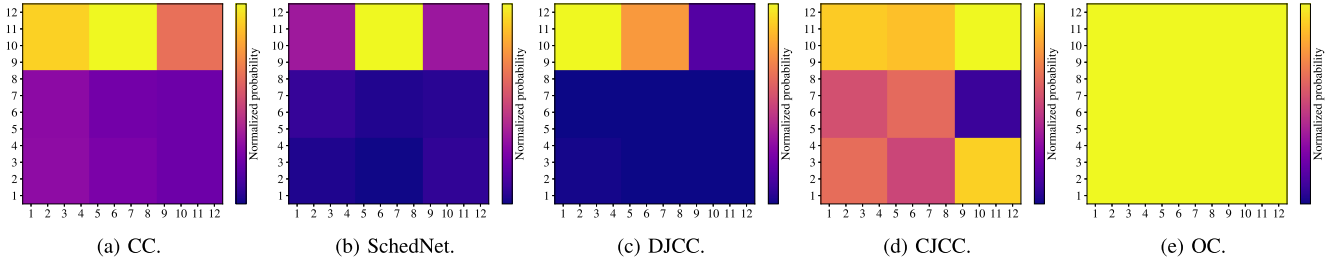


Fig. 9. Heatmap of the transmitted area probability in the data muling task.

can optimize both strategies jointly without any additional design requirements.

We can see the same pattern in Fig. 7(a), which shows the transmission patterns: while the buoys closer to the center of the map transmit much more often when using the CC scheme, buoys that actually have relevant information are much more active in the more advanced schemes, helping the AUV reach the target faster. In particular, the buoys in the JCC schemes have identified the correct VoI function and are able to optimize it, leading to better outcomes for the robot. Finally, Fig. 8 shows the distribution of the AUV positions in the data muling scenario, while Fig. 9 does the same for the buoys transmissions. As for the previous case, we can appreciate that the AUV takes more complex trajectories in

the case of SchedNet (Fig. 8(b)–8(d)) and the JCC approaches (Figs. 8(c)–8(d)) than using the CC strategy (Fig. 8(a)). In this case, there is still a gap between the JCC strategies and the OC scheme, as coordination is much harder, and the trajectories for CJCC are much more evenly distributed than for DJCC.

Looking at the transmission distribution in Figs. 9(b)–9(d) we can get a better comprehension of the performance gap among the JCC schemes. While centralized communication can always deliver relevant information, the buoys in the DJCC scenario cannot coordinate effectively when the AUV is in the lower portion of the map, i.e., at the beginning of the task, when neither node has been visited. Particularly, in order to avoid collisions, which are heavily penalized by the reward function, the buoys tend to only transmit when the AUV is

already close to the target, helping it reach the second node when necessary. Interestingly, SchedNet shows a transmission distribution very similar to CC, suggesting that the learned policy is sub-optimal from a communication perspective if compared to CJCC.

The results reported in this section were deliberately focused on simple use cases, where communication and control performance depends only on the buoys' ability to provide the most accurate information to the AUV, which must adapt its behavior to the received information. Considering more advanced models or real-world applications would increase the number of confounding factors, including communication interference, imperfections in the environment sensing, or inherent errors in the control actions. Such factors complicate the CP-POMDP environment but do not reduce the need for jointly modeling communication and control policies, as the basic features of the problem do not depend on the training complexity. The joint optimization of communication and control decisions involves aspects of game theory that must be handled appropriately to achieve stable solutions. The central aim of this work is to develop a formal framework that accurately models these aspects, offering insights into their impact on communication and control strategies. By employing synthetic data generated through simulations we mitigate the inherent uncertainties present in real-world experiments, thus facilitating greater control over the experimental environment and, consequently, improving the interpretability of the results. Hence, while validation of the proposed CP-POMDP model under real settings will be an interesting research step for the future, the analysis of baseline problems is vital for highlighting the specific characteristics of CP-POMDPs.

VI. CONCLUSION AND FUTURE WORK

In this paper, we developed a new theoretical framework to model communication among multiple agents operating in the same learning environment. Our system, named CP-POMDP, includes two classes of agents: the robots, which have to perform a physical task, and the sensors, which can sense the environment and transmit information to the robots. Since their reward depends directly on the robot's actions, the sensors are encouraged to identify the best communication policy to help the robots complete their task as effectively as possible. The model is extremely flexible and can represent a plethora of practical applications, while still limiting complexity. We tested the described framework in a reference scenario, in which an AUV needs to complete a mission in an underwater environment, while a set of buoys can transmit information about other areas of the map, widening its perception. Our simulations show that the joint training of the buoys and the AUV, based on our CP-POMDP model, significantly outperforms classical networked control strategies.

Future work on the subject might involve more complex scenarios with multiple robots and sensors, in which information is distributed across the agents, none of which has a full view of the scenario. This leads to interesting connections to open machine learning problems, as agents have to develop a theory of mind to estimate not just the state, but also the

knowledge of other agents, and to game theoretical issues, as the behavior of the swarm of robots as a whole is determined by game theoretical principles. The case in which agents have partially conflicting objectives is particularly interesting, as sensors might find it beneficial to transmit false or misleading information, introducing questions of trust and reciprocity.

APPENDIX

Debris avoidance: In the debris avoidance scenario, the obstacles are arranged in three horizontal lines on the map, as shown in Fig. 2(a). Each line of debris has a single opening, whose location can change over time. We denote the coordinates of the free passages on the x -axis at slot k by $z_0(k)$, $z_1(k)$, and $z_2(k)$, respectively. At the beginning of slot k , the set of positions containing obstacles is:

$$M(k, \mathbf{x}) = \omega \iff x^{(1)}(k) \neq z_j^{(1)}(k), x^{(2)}(k) \in \mathcal{Y}_j, \quad (16)$$

where $\mathcal{Y}_j$ is the set containing the coordinates of the j -th debris line on the y axis:

$$\mathcal{Y}_0 = \{2, 3\}, \mathcal{Y}_1 = \left\{ \frac{N}{2}, \frac{N}{2} + 1 \right\}, \mathcal{Y}_2 = \{N - 2, N - 1\}. \quad (17)$$

At the beginning of each episode, the free passages' coordinates are generated from a discrete uniform distribution in $\mathcal{N}$. The locations $z_0(k)$, $z_1(k)$, and $z_2(k)$ are then updated following the same periodicity as the buoys' communication. Hence, at the beginning of any slot k such that $\text{mod}(k, K_p) = 0$, the new values of $z_0(k)$, $z_1(k)$, and $z_2(k)$ are picked from the distribution

$$P(z_j(k) = z) = \frac{(N - |z - z_j(k-1)|)^2}{\sum_{q=1}^N (N - |q - z_j(k-1)|)^2}. \quad (18)$$

The AUV receives a reward $\rho \in \mathbb{R}^+$ when it reaches the surface vessel, i.e., if $\mathbf{x}_\ell(k+1) = \mathbf{x}_v$. Besides, we also assign a smaller, positive intermediate reward $\sigma < \rho$ if the AUV makes progress in its task. The AUV's advancements are determined by the function $\chi(s(k), a_\ell(k))$, which returns 1 if any of the following conditions are true and 0 otherwise:

- 1) If $a_\ell(k) = (0, 1)$ and $M(k, \mathbf{x}_\ell(k) + a_\ell(k)) = \phi$, i.e., if the AUV moves upward in the map, without entering in the area occupied by the debris;
- 2) If $|\mathbf{x}_\ell^{(1)}(k) + a_\ell^{(1)}(k) - z_j(k)| < |\mathbf{x}_\ell^{(1)}(k) - z_j(k)|$ and $\mathbf{x}_\ell^{(2)}(k) + 1 \in \mathcal{Y}_j$, i.e., if the AUV has reached a debris line, and moves horizontally towards a free passage;
- 3) If $\|\mathbf{x}_\ell(k) + a_\ell(k) - \mathbf{x}_f\|_2 < \|\mathbf{x}_\ell(k) - \mathbf{x}_f\|_2$ and $x_\ell^{(2)}(k) = N$, i.e., if the AUV is on the top row of the map, and moves horizontally towards the vessel.

Data muling: In the data muling scenario, we define an *active target* as any target node that has not been visited by the AUV yet. We will only consider active targets, as the ones that have already been visited do not affect the reward and are removed from $\mathbf{M}(k)$. The set of active targets in slot k is denoted by $\mathcal{T}(k)$. At the beginning of each episode, the targets' coordinates are generated from a

discrete uniform distribution in $\mathcal{M}$. The position of the i -th target $\mathbf{t}_i(k)$ then changes following the same periodicity as the buoys' communications. Whenever $\text{mod}(k, K_p) = 0$, the target's position is updated according to the following distribution:

$$P(\mathbf{t}_i(k+1) = \mathbf{x}) = \frac{\prod_{j=1}^2 \left(N - |x^{(j)} - t_i^{(j)}(k)| \right)^2}{\sum_{n \in \mathcal{N}} \left(N - |n - x^{(j)}| \right)^2}, \quad (19)$$

where $x^{(j)}$ is the j -th coordinate of $\mathbf{x}$. In this case, the intermediate reward is tuned to make the AUV visit all targets, and then reach the vessel in the shortest possible time. Practically, the function $\chi(s(k), a_\ell(k))$ is equal to 1 if any of the following conditions are true and 0 otherwise:

- 1) If $|\mathcal{T}(k)| = 2$, $\|\mathbf{x}_\ell(k) + a_\ell(k) - \mathbf{t}_i(k)\|_2 < \|\mathbf{x}_\ell(k) - \mathbf{t}_i(k)\|_2$, and $\|\mathbf{x}_\ell(k) - \mathbf{t}_{3-i}(k)\|_2 + \|\mathbf{t}_{3-i}(k) - \mathbf{x}_v\|_2 < \|\mathbf{x}_\ell(k) - \mathbf{t}_{3-i}(k)\|_2 + \|\mathbf{t}_{3-i}(k) - \mathbf{x}_v\|_2$, with $i \in \{1, 2\}$, i.e., if both targets are still unvisited, the AUV is moving towards $\mathbf{t}_i(k)$, and it is more convenient to visit the node in $\mathbf{t}_i(k)$ to end the mission in the shortest time;
- 2) If $|\mathcal{T}(k)| = 1$ and $\|\mathbf{x}_\ell(k) + a_\ell(k) - \mathbf{t}_1(k)\|_2 < \|\mathbf{x}_\ell(k) - \mathbf{t}_1(k)\|_2$, i.e., if only a single node is still unvisited and the AUV is moving towards its position;
- 3) If $|\mathcal{T}(k)| = 0$ and $\|\mathbf{x}_\ell(k) + a_\ell(k) - \mathbf{x}_f\|_2 < \|\mathbf{x}_\ell(k) - \mathbf{x}_f\|_2$, i.e., if all the targets have been visited and the AUV is moving towards the vessel.

REFERENCES

- [1] J. Cheng, W. Chen, F. Tao, and C.-L. Lin, "Industrial IoT in 5G environment towards smart manufacturing," *J. Ind. Inf. Integr.*, vol. 10, pp. 10–19, Jun. 2018.
- [2] M. Wollschläger, T. Sauter, and J. Jasperneite, "The future of industrial communication: Automation networks in the era of the Internet of Things and industry 4.0," *IEEE Ind. Electron. Mag.*, vol. 11, no. 1, pp. 17–27, Mar. 2017.
- [3] D. Zhang, P. Shi, Q.-G. Wang, and L. Yu, "Analysis and synthesis of networked control systems: A survey of recent advances and challenges," *ISA Trans.*, vol. 66, pp. 376–392, Jan. 2017.
- [4] M. Klügel, M. H. Mamduhi, S. Hirche, and W. Kellerer, "AoI-penalty minimization for networked control systems with packet loss," in *Proc. Int. Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, 2019, pp. 189–196.
- [5] S. Kaul, R. Yates, and M. Gruteser, "Real-time status: How often should one update?" in *Proc. Int. Conf. Comput. Commun. (INFOCOM)*, 2012, pp. 2731–2735.
- [6] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015.
- [7] O. Ayan, M. Vilgelm, M. Klügel, S. Hirche, and W. Kellerer, "Age-of-information vs. value-of-information scheduling for cellular networked control systems," in *Proc. 10th Int. Conf. Cyber-Phys. Syst. (CPS/IT)*, 2019, pp. 109–117.
- [8] C. E. Shannon, "A mathematical theory of communication," *Bell Syst. Tech. J.*, vol. 27, no. 3, pp. 379–423, Jul. 1948.
- [9] D. Ritchie, "Shannon and Weaver: Unravelling the paradox of information," *Commun. Res.*, vol. 13, no. 2, pp. 278–298, Apr. 1986.
- [10] T.-Y. Tung, S. Kobus, J. P. Roig, and D. Gündüz, "Effective communications: A joint learning and communication framework for multi-agent reinforcement learning over noisy channels," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 8, pp. 2590–2603, Aug. 2021.
- [11] A. Lazaridou and M. Baroni, "Emergent multi-agent communication in the deep learning era," 2020, *arXiv:2006.02419*.
- [12] X. Zheng, S. Zhou, and Z. Niu, "Urgency of information for context-aware timely status updates in remote control systems," *IEEE Trans. Wireless Commun.*, vol. 19, no. 11, pp. 7237–7250, Nov. 2020.
- [13] D. Kim et al., "Learning to schedule communication in multi-agent reinforcement learning," in *Proc. 7th Int. Conf. Learn. Represent. (ICLR)*, 2019, pp. 1–17.
- [14] L. Zhang, Y. Shi, T. Chen, and B. Huang, "A new method for stabilization of networked control systems with random delays," *IEEE Trans. Autom. Control*, vol. 50, no. 8, pp. 1177–1181, Aug. 2005.
- [15] Q. Fu, A. Song, F. Zhang, and M. Pan, "Reinforcement learning-based trajectory optimization for data muling with underwater mobile nodes," *IEEE Access*, vol. 10, pp. 38774–38784, 2022.
- [16] H. S. Witsenhausen, "A counterexample in stochastic optimum control," *SIAM J. Control*, vol. 6, no. 1, pp. 131–147, Feb. 1968.
- [17] S. Mitter and A. Sahai, "Information and control: Witsenhausen revisited," in *Proc. Learn., Control Hybrid Syst.*, 2007, pp. 281–293.
- [18] M. Y. I. Zia, J. Poncela, and P. Otero, "State-of-the-art underwater acoustic communication modems: Classifications, analyses and design challenges," *Wireless Pers. Commun.*, vol. 116, no. 2, pp. 1325–1360, Jan. 2021.
- [19] A. Kosta, N. Pappas, and V. Angelakis, "Age of information: A new concept, metric, and tool," *Found. Trends Netw.*, vol. 12, no. 3, pp. 162–259, Nov. 2017.
- [20] R. D. Yates, Y. Sun, D. R. Brown, S. K. Kaul, E. Modiano, and S. Ulukus, "Age of information: An introduction and survey," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 5, pp. 1183–1210, May 2021.
- [21] A. Maatouk, S. Kriouile, M. Assaad, and A. Ephremides, "The age of incorrect information: A new performance metric for status updates," *IEEE/ACM Trans. Netw.*, vol. 28, no. 5, pp. 2215–2228, Oct. 2020.
- [22] F. Mason, M. Giordani, F. Chiariotti, A. Zanella, and M. Zorzi, "An adaptive broadcasting strategy for efficient dynamic mapping in vehicular networks," *IEEE Trans. Wireless Commun.*, vol. 19, no. 8, pp. 5605–5620, Aug. 2020.
- [23] J. Lam, H. Gao, and C. Wang, "Stability analysis for continuous systems with two additive time-varying delay components," *Syst. Control Lett.*, vol. 56, no. 1, pp. 16–24, Jan. 2007.
- [24] D. Yue, Q.-L. Han, and J. Lam, "Network-based robust H_∞ control of systems with uncertainty," *Automatica*, vol. 41, no. 6, pp. 999–1007, Jun. 2005.
- [25] K. Liu, E. Fridman, and L. Hetel, "Networked control systems in the presence of scheduling protocols and communication delays," *SIAM J. Control Optim.*, vol. 53, no. 4, pp. 1768–1788, Aug. 2015.
- [26] F. Chiariotti, A. A. Deshpande, M. Giordani, K. Antonakoglou, T. Mahmoodi, and A. Zanella, "QUIC-EST: A QUIC-enabled scheduling and transmission scheme to maximize VoI with correlated data flows," *IEEE Commun. Mag.*, vol. 59, no. 4, pp. 30–36, May 2021.
- [27] E. Fridman and M. Dambrine, "Control under quantization, saturation and delay: An LMI approach," *Automatica*, vol. 45, no. 10, pp. 2258–2264, Oct. 2009.
- [28] L. X. Zhang, Z. P. Ning, and W. X. Zheng, "Observer-based control for piecewise-affine systems with both input and output quantization," *IEEE Trans. Autom. Control*, vol. 53, no. 4, pp. 5858–5865, Nov. 2017.
- [29] C. Wu, J. Liu, X. Jing, H. Li, and L. Wu, "Adaptive fuzzy control for nonlinear networked control systems," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 47, no. 8, pp. 2420–2430, Aug. 2017.
- [30] Y. Zhang, J. Liu, and X. Ruan, "Iterative learning control for uncertain nonlinear networked control systems with random packet dropout," *Int. J. Robust Nonlinear Control*, vol. 29, no. 11, pp. 3529–3546, Jul. 2019.
- [31] H. Xu, Q. Zhao, and S. Jagannathan, "Finite-horizon near-optimal output feedback neural network control of quantized nonlinear discrete-time systems with input constraint," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 26, no. 8, pp. 1776–1788, Aug. 2015.
- [32] J. N. Foerster, Y. M. Assael, N. de Freitas, and S. Whiteson, "Learning to communicate with deep multi-agent reinforcement learning," in *Proc. 30th Int. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2016, pp. 2145–2153.
- [33] C. Zhang and V. Lesser, "Coordinating multi-agent reinforcement learning with limited communication," in *Proc. Int. Conf. Auton. Agents Multi-Agent Syst. (AAMAS)*, 2013, pp. 1101–1108.
- [34] A. Oroojlooy and D. Hajinezhad, "A review of cooperative multi-agent deep reinforcement learning," *Appl. Intell.*, vol. 53, no. 11, pp. 13677–13722, Jun. 2023.
- [35] C. J. Watkins and P. Dayan, "Q-learning," *Mach. Learn.*, vol. 8, nos. 3–4, pp. 279–292, 1992.
- [36] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. Conf. Artif. Intell.*, vol. 30, 2016, pp. 2094–2100.
- [37] M. H. M. Alkilabi, A. Narayan, and E. Tuci, "Cooperative object transport with a swarm of e-puck robots: Robustness and scalability of evolved collective strategies," *Swarm Intell.*, vol. 11, no. 3, pp. 185–209, Dec. 2017.

- [38] G. J. Laurent, L. Matignon, and L. Fort-Piat, "The world of independent learners is not Markovian," *Int. J. Knowl.-Based Intell. Eng. Syst.*, vol. 15, no. 1, pp. 55–64, 2011.
- [39] M. L. Littman, "Markov games as a framework for multi-agent reinforcement learning," in *Proc. 11th Int. Conf. Mach. Learn. (ICML)*, 1994, pp. 157–163.
- [40] M. L. Littman, "Value-function reinforcement learning in Markov games," *Cogn. Syst. Res.*, vol. 2, no. 1, pp. 55–66, Apr. 2001.
- [41] J. R. Marden, G. Arslan, and J. S. Shamma, "Cooperative control and potential games," *IEEE Trans. Syst., Man, Cybern., Cybern.*, vol. 39, no. 6, pp. 1393–1407, Dec. 2009.
- [42] S. Tadelis, *Game Theory: An Introduction*. Princeton, NJ, USA: Princeton Univ., Jan. 2013.
- [43] P. Talli, F. Mason, F. Chiariotti, and A. Zanella, "Push-and pull-based effective communication in cyber-physical systems," in *Proc. Int. Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, 2024, pp. 1–7.
- [44] X. Deng, N. Li, D. Mguni, J. Wang, and Y. Yang, "On the complexity of computing Markov perfect equilibrium in general-sum stochastic games," *Nat. Sci. Rev.*, vol. 10, no. 1, Jan. 2023, Art. no. nwac256.
- [45] J. Wang, Y. Zhang, T.-K. Kim, and Y. Gu, "Shapley Q-value: A local reward approach to solve global reward games," in *Proc. Conf. Artif. Intell.*, vol. 34, 2020, pp. 7285–7292.
- [46] S. Jiang, "State-of-the-art medium access control (MAC) protocols for underwater acoustic networks: A survey based on a MAC reference model," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 1, pp. 96–131, 1st Quart., 2018.
- [47] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2017, *arXiv:1412.6980*.
- [48] M. Doniec, I. Topor, M. Chitre, and D. Rus, "Autonomous, localization-free underwater data muling using acoustic and optical communication," in *Proc. 13th Int. Symp. Exp. Robot. (ISER)*, 2012, pp. 841–857.



Federico Mason (Member, IEEE) received the master's degrees in telecommunication engineering and the Ph.D. degree in information engineering from the University of Padova, Italy, in 2018 and 2023, respectively, where he is currently an Assistant Professor with the Department of Information Engineering. In 2022, he spent a period abroad as a Visiting Research Scholar with Scripps Research Translational Institute, La Jolla, CA, USA. In 2023, he worked as a Postdoctoral Researcher with the IRCCS Institute of Neurological Sciences of Bologna, Italy. His main research interests include the development of new computational techniques to process physiological signals and the analysis of the training costs in learning-based network optimization.



goal-oriented communications.

Federico Chiariotti (Member, IEEE) received the Ph.D. degree in information engineering from the University of Padova, Italy, in 2019. He is currently an Assistant Professor with the Department of Information Engineering. From 2020 to 2022, he worked on semantic communication and Age of Information with Aalborg University, Denmark. He has published more than 80 peer-reviewed papers and received the Best Paper Award in four conferences. His research focuses on Age of Information, the use of machine learning in networks, and on



Andrea Zanella (Senior Member, IEEE) received the Laurea degree in computer engineering and the Ph.D. degree from the in 1998 and 2001, respectively, University of Padova, Italy. He is one of the coordinators of the SIGnals and NETworking Research Lab. He is a Full Professor with the Department of Information Engineering, University of Padova. His long-established research activities are in the fields of protocol design, optimization, and performance evaluation of wired and wireless networks.



Petar Popovski (Fellow, IEEE) received the Dipl.-Ing. and M.Sc. degrees in communication engineering from the University of Sts. Cyril and Methodius in Skopje, and the Ph.D. degree from Aalborg University in 2005. He is a Professor with Aalborg University, where he heads the section on Connectivity, and a Visiting Excellence Chair with the University of Bremen. He authored the book *Wireless Connectivity: An Intuitive and Fundamental Guide*. His research interests are in the area of wireless communication and communication theory. He received an ERC Consolidator Grant in 2015, the Danish Elite Researcher Award in 2016, the IEEE Fred W. Ellersick Prize in 2016, the IEEE Stephen O. Rice Prize in 2018, the Technical Achievement Award from the IEEE Technical Committee on Smart Grid Communications in 2019, the Danish Telecommunication Prize in 2020, and the Villum Investigator Grant in 2021. He was a Member at Large at the Board of Governors in IEEE Communication Society from 2019 to 2021. He is currently an Editor-in-Chief of IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS and the Chair of the IEEE Communication Theory Technical Committee.